MarketSim v2.1
A compact market making simulator with market dynamics, risk, telemetry, and outputs

Saxon Lee

12 December 2025

Abstract

This document explains my MarketSim v2.1 end to end. The core is a single C++ sim file (learning C++ FAFO style so be easy on my code quality) that simulates a small limit order book, a market making agent, and an external flow model. The simulator writes telemetry and a trade sim data to CSV. A Streamlit app reads those files and produces plots and diagnostics that match how I would explain a run in an interview.

I'll chat a bit about motivations and outputs. I then walk through the C++ design in detail: data structures, matching rules, tick handling, the agent policy, and the accounting identities. I explain the market dynamics I use, why they matter, and what they stress. I then explain the Streamlit layer and the report builder. Finally, I'll show what I can deduce from one concrete run by reading the telemetry, trades, and the PDF report, and I end with honest differences versus live MM/HFT.

1 Purpose and audience

I built this project to learn market making by building a tight simulation loop. I wanted a world that is small enough to understand line by line, but rich enough to show the right risks. The goal is not a high fidelity exchange model. The goal is an honest sandbox with clear logs.

This write up is for:

- a junior quant reviewer who wants to see clean engineering and sound metrics;
- an interviewer who wants a clear story: what I built, what it measures, and what I learnt.

2 What the simulator produces (source of truth)

A single run produces three artefacts.

1. **Telemetry CSV:** end of step snapshots (one row per step).
   - state: time/step, bid, ask, my quotes, inventory;
   - P&L: cash, realised P&L, unrealised P&L, equity;
   - operations: fills and cancels (cumulative counters);
   - microstructure: spread, imbalance, microprice, time at touch.
2. **Trades CSV:** a trade tape (one row per print). Each row includes side, price, quantity, whether my agent participated, and queue diagnostics.
3. **Summary PDF report:** a fixed set of plots for quick review and for sharing.

Why I treat the C++ outputs as the source of truth. The Streamlit app does not change the data. It only checks, summarises, and plots. If the viewer and the sim disagree, I assume the viewer is wrong until proven otherwise.

3 Code organisation (C++ Simulator)

The code is a single translation unit (`market-sim-v2.1.cpp`). I keep it in one file on purpose. It makes review easy.

At a high level the code has these blocks.

1. **Types and utilities:** tick rounding, RNG helpers, and small structs.
2. **Order book:** price levels and FIFO queues, with add/cancel/match.
3. **Market dynamics:** mid price process and external order flow.
4. **Agent:** quoting logic, inventory skew, size control, and urgent flattening.
5. **Accounting and telemetry:** equity and per step P&L, plus CSV output.
6. **Driver loop:** the step function and the run function.

In the rest of this paper I refer to these as modules, even though they live in one file.

4 Order book model

4.1 Tick grid and price representation

All prices snap to a tick size Δ.

$$p_{\text{tick}} = \Delta \cdot \text{round}\left(\frac{p}{\Delta}\right).$$

I do this at input boundaries: when I create a limit order, when I update the mid, and when I query the best bid/ask.

Reason. A tick grid removes fragile behaviour from floating point keys. It also makes spreads and queue placement easy to reason about.

4.2 Data structures

The book holds two maps:

- bids: price levels sorted high to low;
- asks: price levels sorted low to high.

Each price level stores a FIFO container of orders. An order stores:

- order id;
- side (buy/sell);
- limit price;
- remaining quantity;
- owner tag (external vs my agent);
- time stamp (step index).

Why FIFO. FIFO gives a clear queue position and makes fills explainable.

4.3 Matching rules

A market order walks the opposite side of the book.

- For a buy market order: match against the best ask, then the next ask, and so on.
- For a sell market order: match against the best bid, then the next bid, and so on.

A limit order crosses if it is priced through the spread. In that case it matches immediately and any remainder rests.

A match produces a *fill* record:

- fill price, quantity, aggressor side;

- maker/taker flags;
- whether my agent participated;
- ids of the resting and incoming orders.

The driver appends each fill to `trades_v5.csv`.

5 Market dynamics (theory choices and what they stress)

The simulator needs a mid price and external flow.

5.1 Mid price and top of book

The mid is the reference price used by the agent when it quotes. The visible best bid and ask come from the simulated book.

In v2.1 I treat the mid as an internal "mark" price. It is the value I use for mark to market P&L and markout.

Why a mark. The book can be thin. A simple mid process keeps the accounting stable and lets me measure adverse selection with a consistent reference.

5.2 External order flow

External flow generates:

- limit orders that replenish the book;
- market orders that consume liquidity.

I draw counts from Poisson laws per step. I draw side from a Bernoulli law with optional bias.

What this stresses. It stresses queue risk, inventory risk, and adverse selection, without needing a full agent based market.

5.3 Toxic bursts (regimes)

Real flow is not i.i.d. It clusters. I model this with a simple two state regime.

Let $S_t \in \{N, T\}$ be the regime (normal vs toxic). The regime is a Markov chain with transition matrix

$$\mathbf{P} = \begin{pmatrix} 1-\alpha & \alpha \\ \beta & 1-\beta \end{pmatrix}.$$

When $S_t = T$ I increase the market order rate and I increase side bias. I treat that as "toxic" because fills are more likely to be informed.

What to look for. In the Streamlit app I use markout to show that short horizon fills can be bad even when longer horizon P&L is positive.

5.4 Sweeps and gaps

With small probability per step I simulate a sweep. A sweep is a large aggressive order that removes multiple levels.

Why it matters. It creates gap risk and tests cancel latency and inventory caps.

6 Agent policy

The market making agent is symmetric when inventory is near zero and asymmetric when inventory grows.

6.1 Quoting around a reference

At each step the agent computes a reference price m_t (the mark). It chooses a base half spread h and posts:

$$p_t^{\text{bid}} = m_t - h - \phi\,\tilde{q}_t, \qquad p_t^{\text{ask}} = m_t + h - \phi\,\tilde{q}_t,$$

where $\tilde{q}_t$ is normalised inventory and ϕ is the skew strength.

Interpretation. If I am long, $\tilde{q}_t > 0$ and I shift both quotes down. That makes my ask more aggressive and my bid less aggressive.

6.2 Size control

The agent reduces quote size as inventory grows. This caps inventory variance.

6.3 Minimum quote life and cancel latency

The agent enforces:

- a minimum life before it cancels a resting quote;
- a cancel latency so a cancel is not instant.

This is a simple way to model that you cannot always pull quotes in time.

6.4 Urgent flattening

If $|q_t|$ breaches a threshold, the agent sends a market order to reduce risk. This pays taker fees. It can also reduce tail losses.

7 Accounting and telemetry

7.1 Cash, realised P&L, unrealised P&L

The simulator tracks:

- cash balance;
- realised P&L from round trips;
- inventory q_t and a mark price m_t.

In the sim, the telemetry uses a strict identity:

$$E_t = \texttt{cash}_t + \texttt{inv_value}_t, \tag{1}$$

where `inv_value` is the marked inventory value at `mid_mark`. The file also logs `realized_pnl` and `unreal_pnl` so I can explain P&L composition.

Logging convention. I log one row per step after matching and after fees. This avoids duplicate time stamps and makes risk metrics coherent.

7.2 Per step P&L

Per step P&L is $\Delta E_t = E_t - E_{t-1}$. I treat this as the basic return series for diagnostics.

Sharpe warning. A per step Sharpe is only meaningful if steps map to time. In my runs the step size is sub second, so annualising needs a clear choice. In the Streamlit app I show Sharpe, but I do not rely on it as the headline metric.

8 HFT KPIs and diagnostics

This project is most useful when it explains *why* a run made money. I track:

- inside share (time at best bid/ask);
- fill counts and fill rate;
- realised and unrealised P&L split;
- markout at short horizons (adverse selection);
- drawdown depth and where it occurs;
- cancels submitted vs cancels confirmed.

8.1 Markout and adverse selection

For each fill at time t with price p_t and signed inventory change Δq_t, the horizon-h markout is

$$\Pi_h = \Delta q_t \, (m(t+h) - p_t),$$

where $m(\cdot)$ is `mid_mark` sampled from telemetry. Negative short horizon markouts indicate adverse selection.

9 Python layer: Streamlit telemetry viewer

The file `market_sim_streamlit_telemetry_viewer_v_2.1_compatible.py` is a viewer and report builder.

9.1 Design goals

- load telemetry and trades and produce consistent plots;
- compute robust metrics that do not break on outliers;
- flag implausible outputs early (for example a Sharpe that suggests a logging bug);
- export a PDF report for review.

9.2 Data loading and cleaning

The app parses the CSVs, coerces numeric columns, sorts by time/step, computes derived series (per step P&L, running peak, drawdown), and performs sanity checks (duplicate step checks, missing bid/ask checks, and accounting identities).

9.3 Metrics

The app reports:

- final equity and max drawdown;
- realised vs unrealised split;
- per step Sharpe (with a user set steps per day);
- rolling Sharpe (traditional and robust);
- markout at short horizons;
- simple microstructure summaries (spread distributions, time at touch).

9.4 Plots and PDF report

The app produces a standard set of plots and writes them into a PDF via `PdfPages`. This is the report I attach when I share results.

10 Outputs explained in detail

This section pins down what each output means and how it connects to the code.

10.1 Telemetry schema (end-of-step state)

A single telemetry row is the state after matching and accounting updates for that step.

In the run analysed here, the telemetry file has 3000 steps and 32 columns. The most important fields are:

Field	Meaning
`step`	integer time index (0 to 2999)
`t`	simulation time in seconds (step size 0.1 s)
`bid, ask`	best bid/ask in the book
`mid_mark`	marking mid used for mark-to-market
`equity`	total marked equity
`cash`	cash balance
`inv_value`	marked value of inventory
`realized_pnl`	cumulative realised P&L
`unreal_pnl`	current unrealised P&L
`fees_paid`	cumulative fees paid
`inventory`	signed position
`spread`	top-of-book spread (`ask` − `bid`)
`microprice`	microprice signal
`imbalance`	top-of-book imbalance
`mm_bid_px, mm_ask_px`	my quoted prices
`mm_bid_qty, mm_ask_qty`	my quoted sizes
`mm_fills`	cumulative fill count (not per-step)
`mm_cancels_sub`	cumulative cancel submissions
`mm_cancels_ok`	cumulative cancels confirmed
`mm_at_best_bid, mm_at_best_ask`	whether my quote is best
`tob_share_bid, tob_share_ask`	time-at-touch shares
`last_trade_px, last_trade_sign`	last print in the step

Equity identity check (again, because it matters). A basic sanity check is that equity decomposes as

$$E_t = \texttt{cash}_t + \texttt{inv_value}_t. \tag{2}$$

In the analysed run this holds at every step to floating point tolerance.

10.2 Trades schema (event log)

In the analysed run, the trades file has 35,238 rows. 27,552 are marked `mm_involved`=1.

Field	Meaning
`t`	event time in seconds
`px, qty`	trade price and size (here `qty` = 1 always)
`sign`	trade sign (direction indicator)
`mm_involved`	whether my MM is one side of the trade
`mid_mark`	marking mid at the event
`spread`	book spread at the event
`maker_age_s`	age of maker order at execution
`maker_queue_ahead_qty_entry`	queued quantity ahead at entry

Inventory reconstruction check. If I define per-trade inventory change as

$$\Delta q = -\texttt{sign} \times \texttt{qty} \quad \text{for trades with} \quad \texttt{mm_involved} = 1, \tag{3}$$

then the cumulative sum of Δq matches the telemetry `inventory` path at all telemetry times. This is a strong consistency check across the two outputs.

10.3 Summary PDF report

The PDF report is a fixed set of plots derived from telemetry (plus derived series such as per step P&L and drawdown). In this run it includes:

1. Equity curve with drawdown shading and an annotation for max drawdown.
2. Drawdown over time.
3. Inventory over time.
4. Top-of-book (bid/ask) and marking mid.
5. Spread and microprice diagnostics.
6. P&L components and cash (realised, unrealised, fees, cash).
7. Per-step P&L histogram.
8. Rolling Sharpe (traditional).
9. Rolling Sharpe (robust).
10. Inventory vs next-step P&L scatter.

What the report is for. It is a quick diagnostic. It shows if a run is plausible, if risk is controlled, and if the policy is trading as intended.

11 One run: findings from telemetry and trades

This section writes down what I can deduce from `telemetry_v5.csv`, `trades_v5.csv`, and the PDF report for one run.

11.1 Basic run facts

- Steps: 3000.
- Time span: 299.9 s (step size 0.1 s).
- Total trades: 35,238.
- MM-involved trades: 27,552 (matches the final value of `telemetry.mm_fills`).

11.2 Overall performance and risk

From telemetry:

- Final equity: \$286.1804.

- Realised vs unrealised: \$272.7414 realised and \$7.9286 unrealised.
- Max drawdown: \$89.7284 (matches the report annotation `Max DD: 89.73`).
- Final inventory: +32.

Interpretation. Most of the P&L is realised. The drawdown is material relative to total profit, which is normal for a passive strategy that sometimes carries inventory through adverse moves.

11.3 Per-step P&L shape

Define per-step P&L as $\Delta E_t = E_t - E_{t-1}$. For this run:

- Mean $\mathbb{E}[\Delta E_t] \approx \$0.0954$.
- Standard deviation $\mathrm{sd}(\Delta E_t) \approx \$5.0909$.
- Best step: \$+35.6502; worst step: \$-23.9992.
- Right-skewed, heavy-tailed distribution (skew ≈ 0.72, excess kurtosis ≈ 6.34).

Interpretation. The run is not driven by one lucky spike. Tail events exist and are visible in the per-step P&L histogram.

11.4 What drives drawdown: inventory plus adverse mid move

The largest drawdown occurs at about step 1290 (time 129.1 s). At that point inventory is about +210 shares and `mid_mark` moves against the position.

Interpretation. In this run, inventory risk is the main drawdown driver. This is the correct failure mode for market making: you get paid small amounts often, and you lose when you are caught with inventory during a directional move.

11.5 Fill intensity and per-fill economics

The market maker is involved in 27,552 fills in 299.9 s, which is about 91.87 fills per second. Total P&L divided by fill count is about \$0.01039 per fill.

Interpretation. One cent per fill is the right scale for a one-lot simulator with a one-to-two cent spread. It suggests the strategy is earning through repeated small edges, not through rare jumps.

11.6 Short-horizon adverse selection vs longer-horizon edge (markouts)

For each MM-involved trade with signed inventory change Δq, define horizon-h markout P&L as

$$\Pi_h = \Delta q\,(m(t+h) - p), \tag{4}$$

where p is trade price and $m(\cdot)$ is `mid_mark` sampled from telemetry.

Empirically in this run:

- $h = 0.1$ s: mean ≈ -0.0497, median ≈ -0.06, win rate ≈ 0.230.
- $h = 1$ s: mean ≈ -0.0116, median ≈ -0.01, win rate ≈ 0.447.
- $h = 5$ s: mean $\approx +0.0127$, median $\approx +0.01$, win rate ≈ 0.518.
- $h = 10$ s: mean $\approx +0.0116$, median $\approx +0.01$, win rate ≈ 0.523.

Interpretation. This is a realistic pattern. Very short-horizon markout is negative because informed flow hits resting liquidity. Over a few seconds, the strategy earns back edge through spread capture and mean reversion.

11.7 Quoting behaviour: inside share and distance to touch

From telemetry:

- Mean book spread ≈ 0.01899; the 95th percentile is ≈ 0.05.
- The maker is at best bid about 35.8% of steps and at best ask about 33.7%.
- Median distance from best bid is about one tick; median distance above best ask is about two ticks.

Interpretation. This looks like deliberate skew. The agent is often competitive on one side while stepping back on the other side. That is consistent with inventory control.

11.8 Cancels and operational realism

In this run:

- Fills: 27,552.
- Cancel submissions: 37,688.
- Cancel confirmations: 3,721.

A useful way to read this is:

- Fill-to-submit ratio: $27{,}552/37{,}688 \approx 0.731$.
- Confirmed-to-submitted cancels: $3{,}721/37{,}688 \approx 0.099$.

Interpretation. The gap between "cancel submitted" and "cancel confirmed" is useful. It creates a window where you can be filled after you try to cancel. That is one of the real risks in passive market making.

12 Sharpe: what it means here and what it does not mean

Sharpe is included because it is familiar, but its meaning depends on the time mapping. In this simulator, one step is 0.1 s. If I annualise Sharpe using an arbitrary "steps per day", I can produce numbers that look impressive and mean nothing.

What I do instead. I treat Sharpe as a relative diagnostic across runs with the same clock and parameters. For explanation and interviews, I prefer:

- P&L per fill,
- markouts at several horizons,
- max drawdown and drawdown duration,
- inventory distribution and tail exposures.

13 Build and run

Compile the C++ simulator (MSYS2 UCRT64)

```
# build
g++ -std=c++20 -O2 -DNDEBUG -Wall -Wextra -Wconversion \
  -o market-sim-v2.1.exe market-sim-v2.1.cpp

# run (example)
./market-sim-v2.1.exe --steps 3000 --seed 42
```

Run the Streamlit app (Windows + venv)

```
# create venv using Windows Python (recommended)
py -3.12 -m venv .venv

# activate
source .venv/Scripts/activate

# upgrade packaging tools
python -m pip install --upgrade pip setuptools wheel

# install deps
pip install streamlit pandas numpy matplotlib seaborn scipy

# run app
streamlit run market_sim_streamlit_telemetry_viewer_v_2.1_compatible.py
```

Note on MSYS2 Python. MSYS2 Python can hit SSL and build tool issues when pip needs to compile wheels. Using the Windows Python launcher (`py`) avoids that. I keep the C++ build in MSYS2, but I run Streamlit from a standard Windows venv.

14 How to read one run

I use a fixed order.

1. Equity and drawdown: note the deepest dip and recovery.
2. Inventory: check if drawdowns line up with large inventory.
3. Markout: check if short horizon markout is negative.
4. Fill counts: confirm the strategy earns via many small edges.
5. Outliers: inspect the worst steps and the trade context.

15 What I learnt (v2.1)

- Clean logging matters more than fancy metrics. One snapshot per step fixes a lot.
- The main risk is inventory through adverse drift. That shows up in drawdown.
- Short horizon adverse selection can coexist with positive longer horizon edge.
- A viewer should include plausibility checks. Otherwise you can fool yourself.

16 Differences vs live MM/HFT (honest critique)

This simulator is for learning. Live market making has constraints and frictions that matter.

16.1 Latency and queue dynamics are simplified

I log maker age and queued quantity ahead, but the matching model is still a simplified FIFO world. In live markets, queue priority depends on venue rules (order modify behaviour, hidden liquidity, partial cancels, and exchange-specific details).

16.2 The order flow model is not calibrated

The arrival process and price dynamics are designed to stress behaviours (adverse selection bursts, inventory carry, spread widening). A live model needs calibration to real trade intensity, spread distributions, and event clustering.

16.3 Fees and rebates are stylised

Real fee schedules are tiered and depend on venue, volume, and order type. A live strategy is often a fee-engineering problem as much as a prediction problem.

16.4 No hard operational limits

Real trading has risk limits, kill switches, throttles, and compliance constraints. A production policy must respect these even under failure modes (data dropouts, feed delays, partial connectivity).

16.5 No cross-venue effects

This is a single venue world. In reality, best execution and toxicity are linked across venues via routing and arbitrage.

Why these gaps are acceptable. The goal is not to pretend this is production. The goal is to show that I understand the core P&L mechanics, risk, and microstructure diagnostics well enough to reason about what would break and why.

Appendix A: metrics used in this paper

$$
\begin{aligned}
E_t &= \texttt{cash}_t + \texttt{inv_value}_t \\
\Delta E_t &= E_t - E_{t-1} \\
\text{Drawdown}_t &= \max_{u \le t} E_u - E_t \\
\Delta q &= -\texttt{sign} \cdot \texttt{qty} \quad (\text{for } \texttt{mm_involved} = 1) \\
\Pi_h &= \Delta q \, (m(t+h) - p) \\
\text{PnL per fill} &= \frac{E_T - E_0}{\#\{\text{MM-involved fills}\}}
\end{aligned}
$$

Appendix B: citations and theory sources

This project was built from notes and ideas from standard market microstructure and market making sources. In the next revision of this paper I will list each book and map each model choice to the chapter that motivated it.